

Cicli for

Cicli for

Come ripetere un'azione per ogni elemento di una lista o sequenza.

Perché usare i cicli?

Immagina di dover salutare 100 studenti uno per uno. Senza i cicli, dovresti scrivere `print("Ciao!")` cento volte. Con un ciclo, lo scrivi una volta sola e Python lo ripete quante volte vuoi.

Il ciclo `for` in particolare è perfetto quando sai già **cosa** vuoi scorrere: una lista di nomi, le lettere di una parola, i numeri da 1 a 10.

Come funziona

```
for elemento in sequenza:  
    # codice da eseguire per ogni elemento
```

Ad ogni giro del ciclo, la variabile `elemento` assume il valore successivo della sequenza. Quando la sequenza finisce, il ciclo si ferma.

Scorrere una lista

L'uso più comune: fare qualcosa per ogni elemento di una lista.

```
frutta = ["mela", "banana", "ciliegia"]  
  
for frutto in frutta:  
    print(frutto)
```

Output:

```
mela  
banana  
ciliegia
```

La variabile `frutto` (che hai scelto tu) prende il valore `"mela"` al primo giro, `"banana"` al secondo, `"ciliegia"` al terzo.

Scorrere una stringa

Una stringa è una sequenza di caratteri, quindi puoi scorrerne le lettere una ad una:

```
for carattere in "Python":  
    print(carattere)
```

Output:

```
P  
y  
t  
h  
o  
n
```

Ripetere un numero preciso di volte con `range()`

Se vuoi eseguire qualcosa esattamente N volte, usa `range()` :

```
for i in range(5):  
    print("Ciao!")
```

Questo stampa "Ciao!" 5 volte. La variabile `i` vale 0, poi 1, poi 2, poi 3, poi 4.

Vedi la sezione su `range()` per tutti i dettagli su come usarla.

Sapere anche la posizione con `enumerate()`

A volte, mentre scorri una lista, vuoi sapere sia il valore sia la sua posizione (indice). Usa `enumerate()` :

```
frutta = ["mela", "banana", "ciliegia"]

for indice, frutto in enumerate(frutta):
    print(f"{indice}: {frutto}")
```

Output:

```
0: mela
1: banana
2: ciliegia
```

Puoi anche scegliere da quale numero partire:

```
for numero, frutto in enumerate(frutta, start=1):
    print(f"{numero}. {frutto}")
```

Output:

```
1. mela
2. banana
3. ciliegia
```

Scorrere due liste insieme con `zip()`

`zip()` abbina gli elementi di due liste, uno a uno:

```
nomi = ["Alice", "Bob", "Carlo"]
voti = [8, 7, 9]

for nome, voto in zip(nomi, voti):
    print(f"{nome} ha preso {voto}")
```

Output:

```
Alice ha preso 8
Bob ha preso 7
Carlo ha preso 9
```

Interrompere il ciclo: **break** e **continue**

break — ferma il ciclo immediatamente:

```
for i in range(10):
    if i == 5:
        break          # quando i vale 5, esci dal ciclo
    print(i)
# stampa: 0, 1, 2, 3, 4
```

continue — salta questo giro e va al prossimo:

```
for i in range(10):
    if i % 2 == 0:
        continue      # se il numero è pari, saltalo
    print(i)
# stampa solo i numeri dispari: 1, 3, 5, 7, 9
```

Cicli dentro cicli (annidati)

Puoi mettere un ciclo **for** dentro un altro. Per esempio, per costruire una tabellina:

```
for i in range(1, 4):
    for j in range(1, 4):
        print(f"{i} x {j} = {i * j}")
    print("----")
```

Output:

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
----
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
----
...
```

Scorrere un dizionario

I dizionari possono essere scorsi in modi diversi:

```
persona = {"nome": "Alice", "eta": 16, "citta": "Roma"}

# Solo le chiavi
for chiave in persona:
    print(chiave)    # nome, eta, citta

# Chiavi e valori insieme
for chiave, valore in persona.items():
    print(f"{chiave}: {valore}")
```